INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABORATORY

PROJECT TITLE: LIBRARY CIRCULATION AND DEMAND DASHBOARD

NAME: 1. Kushal Chakraborty

ROLL NO: 58

ENROLLMENT NO: 12025052003063

SECTION: A

SUBMITTED TO: Prof. Indranil Basu

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINEERING

ACADEMIC SESSION: 2026-2027

DATE OF SUBMISSION: 10 November 2026

Abstract

This project develops a Python program that analyses the borrowing records of a campus library to identify which book categories are in demand and how reliable that conclusion is. It validates records, converts them into objects of a *LoanRecord* class, stores them in a *LoanCollection* container, summarises loan lengths with NumPy, aggregates demand by category and month with Pandas, exports JSON/CSV summaries, and draws four labelled Matplotlib figures.

The demonstration data are **simulated**, not real library records. From 437 input rows, 420 pass validation and 17 are rejected (8 duplicates, 4 incomplete rows, 3 with a return date before the issue date and 2 with an impossible date). By loan count the most requested category is Computer Science (102 loans, 24.3%), almost tied with Electronics (101). Per catalogue title, however, Mathematics is busiest (2.10 loans per title), so the answer depends on how demand is defined. Loans last 11.1 days on average. 24.0% of loans are overdue under the stored 14-day rule, but only 2.1% if a 7-day grace period is allowed. The results demonstrate the dashboard's behaviour and do not describe real students.

Keywords

Library circulation; demand analysis; overdue loans; data validation; Python; NumPy; Pandas; Matplotlib; object-oriented programming; CSV; JSON.

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Software and System Requirements
4. System Design and Methodology
5. Implementation of Major Modules
6. Results and Discussion
7. Testing and Validation
8. Limitations and Future Scope
9. Conclusion
10. References
11. Appendix: CSV Format, Execution and Contributions

1. Introduction

A campus library records every loan: which book, which borrower, when it was issued, when it was due and when it came back. Stored as a spreadsheet, these records are hard to read at a glance, and they often contain mistakes such as duplicated lines, blank fields or impossible dates. If such errors are not removed, statistics about demand can be misleading, and decisions such as which subjects need more copies may be wrong.

The developed program reads a loan file, validates each row, and reports demand by category and by month, loan durations and overdue rates. It also tests whether its own conclusions are stable: it compares results before and after data cleaning, compares two definitions of demand, and compares two overdue rules. All figures are produced from a simulated dataset that is clearly labelled as such.

2. Problem Statement and Objectives

Problem statement: Analyse borrowing records and book categories for a campus library. Use functions for overdue checks and frequency counts; store loans in CSV; implement a *LoanRecord* class; use NumPy for loan durations; use Pandas for category and month summaries and JSON export; draw trends and category bars. Identify high-demand categories while checking duplicated and incomplete records.

The principal objectives are:

- Validate every record and reject duplicates, incomplete rows, impossible dates and unknown categories, while reporting the reason for each rejection.
- Represent loans with a *LoanRecord* class and a *LoanCollection* container that use special methods.
- Compute loan-duration statistics and a moving average of monthly loans with NumPy.
- Summarise demand by category and month with Pandas and export the results as JSON and CSV.
- Draw labelled plots of monthly trend, category demand, loan-length distribution and the evaluation results.
- Evaluate whether the conclusions change when (a) uncleaned data are used, (b) demand is measured per catalogue title instead of by raw count, and (c) a grace period is added to the overdue rule.

3. Software and System Requirements

Component	Purpose
Python 3.9 or newer	Program execution
csv, json, datetime, collections, argparse, unittest	File input/output, dates, frequency counts, command-line options and testing
NumPy	Duration arrays, percentiles, standard deviation and moving average
Pandas	DataFrames, group-by summaries, ranking and CSV/JSON export
Matplotlib	Bar charts, trend line and histogram (saved as PNG files)
Python IDLE / VS Code / terminal	Running the program: *python library_dashboard.py*

Any desktop computer with Python and the three third-party libraries is sufficient. The program writes image files and needs no graphical window, so it also runs on a server or over SSH; the option *--show* opens the plot windows.

4. System Design and Methodology

4.1 Input data and schema

Each loan has seven fields: *loan_id* (unique), *student_id* (anonymised), *book_id*, *category*, *issue_date*, *due_date* and *return_date* (blank if the book is still on loan). A second file, *catalogue.csv*, lists every book title with its category (240 titles in six categories) and is used to measure demand per title.

4.2 Simulated data

No real library data were used. *data_generator.py* creates the files with a fixed random seed (42), so the results are reproducible. It generates 420 valid loans over the academic year August 2025 to July 2026 for 80 anonymised students (eight of them borrow much more often) and then inserts 17 deliberately faulty rows so that the validation code has something to detect. The generator gives each category a different demand weight, gives December and May (exam months) more loans, and makes exam-time loans slightly longer. **Because these patterns were built into the generator, finding them again shows that the pipeline works; it is not a discovery about real borrowers.**

4.3 Validation rules

- All mandatory fields (everything except *return_date*) must be non-blank.
- Dates must be valid calendar dates in YYYY-MM-DD form.
- The due date and the return date must not be earlier than the issue date.
- The category must exist in the catalogue.
- *loan_id* must be unique; later repeats are rejected as duplicates.

4.4 Analysis workflow

1. Read the CSV into a list of dictionaries (*csv.DictReader*) and check the column names.
2. Validate each row, build *LoanRecord* objects and log every rejection with its line number and reason.
3. Sort the records chronologically and place them in a *LoanCollection*.
4. Compute loan-length statistics with NumPy and convert the collection to a Pandas DataFrame.
5. Summarise by category and by month; compute overdue rates and loans per title.
6. Evaluate the alternatives (raw vs cleaned counts, count vs per-title demand, 0-day vs 7-day grace).
7. Export JSON/CSV summaries, draw the four figures and print a written interpretation.

4.5 Definitions and measures

Measure	Definition
Loan length	Return date minus issue date in days (returned loans only for statistics)
Overdue	Return date (or, if still on loan, 31 July 2026) is later than the due date by more than the grace period; the due date is 14 days after issue
Demand (count)	Number of valid loans in a category
Demand (per title)	Loans divided by the number of catalogue titles in the category
Peak month	Month whose loan count exceeds the monthly mean plus one standard deviation
3-month moving average	Mean of each three consecutive monthly counts (NumPy convolution)

Still-open loans are counted as overdue if their due date has passed at the reference date, but they are left out of loan-length statistics because their final length is unknown. The reference date is fixed in the code, so results do not depend on the computer's clock.

5. Implementation of Major Modules

Module / function	Functionality
parse_date(), is_overdue(), frequency_counts()	Date parsing, overdue rule with grace period, sorted frequency counts
LoanRecord (from_dict, loan_days, overdue, to_dict)	One validated loan; cannot exist in an invalid state
LoanCollection (durations, to_dataframe)	List-like container; hands data to NumPy and Pandas
read_csv_rows(), load_catalogue(), validate_rows()	File input, column checking, validation and rejection log

Module / function	Functionality
duration_statistics(), moving_average(), monthly_counts()	NumPy statistics and monthly series
category_summary(), evaluate()	Pandas group-by summary and the three evaluations
interpretation()	Automatic written discussion with a simulated-data warning
plot_monthly_trend(), plot_categories(), plot_durations(), plot_evaluation()	The four Matplotlib figures
run(), main()	Pipeline, console report, exports and command-line options

5.1 Classes and special methods

Special method	Class	Why it is used
__init__	LoanRecord	Validates fields on creation, so invalid records cannot be created
__repr__	LoanRecord, LoanCollection	Readable printing and debugging output
__eq__, __hash__	LoanRecord	Two records are the same loan if *loan_id* matches; sets and dicts can detect duplicates
__lt__	LoanRecord	*sorted(records)* gives a chronological list
__len__	LoanCollection	*len(collection)* returns the number of loans
__getitem__	LoanCollection	Indexing and slicing (a slice returns a new collection)
__iter__, __contains__	LoanCollection	*for* loops, DataFrame construction and *in* tests

5.2 Course-outcome and Bloom-level coverage

Outcome	Where it appears in the project
CO1 control flow and functions	is_overdue, parse_date, frequency_counts, validate_rows
CO2 data structures and file I/O	Dictionaries, lists, sets, Counter, csv and json modules
CO3 classes and special methods	LoanRecord and LoanCollection (Section 5.1)
CO4 NumPy	Duration arrays, percentiles, standard deviation, moving average
CO5 Pandas and CSV/JSON	Group-by summaries, ranking, CSV and JSON export
CO6 Matplotlib and interpretation	Figures 1 to 4 and the written interpretation (Section 6.6)
BL2 explain	Schema, rules and plots are explained in Sections 4 and 6
BL3 implement	Working pipeline run from one command
BL4 analyse	Failure-case analysis in Sections 6.1 and 7
BL5 evaluate	Three alternatives compared in Section 6.5
BL6 create	Complete reproducible application with data generator, tests and README

6. Results and Discussion

6.1 Data quality

The following results come from the fixed-seed simulated dataset in the submitted repository. They change if another CSV file is supplied.

Indicator	Observed value
Input rows	437
Valid records	420

Indicator	Observed value
Rejected rows	17
Rejected: duplicate loan_id	8
Rejected: missing mandatory field	4
Rejected: return date before issue date	3
Rejected: invalid date format	2
Loans still on loan at 31 Jul 2026	3
Mean / median loan length	11.1 / 11 days
Standard deviation / 90th percentile	4.7 / 17 days
Overdue loans (14-day rule)	24.0%

All 17 faulty rows were detected, and each was assigned to the correct reason. Without cleaning, the category counts would be too high by 0 to 6% (Section 6.5).

6.2 Monthly trend

Monthly loans average 35.0. December 2025 (57 loans) and May 2026 (47) exceed the mean-plus-one-standard-deviation threshold and are flagged as peaks. In the simulation these are the exam months, so the peaks are expected; in real data the flagged months would be compared with the academic calendar and the data-collection record before being called genuine demand rather than incomplete or duplicated data. August 2025 (18) and July 2026 (17) are the lowest months. The moving average smooths the series: it rises to its highest value around November and then declines through early 2026.

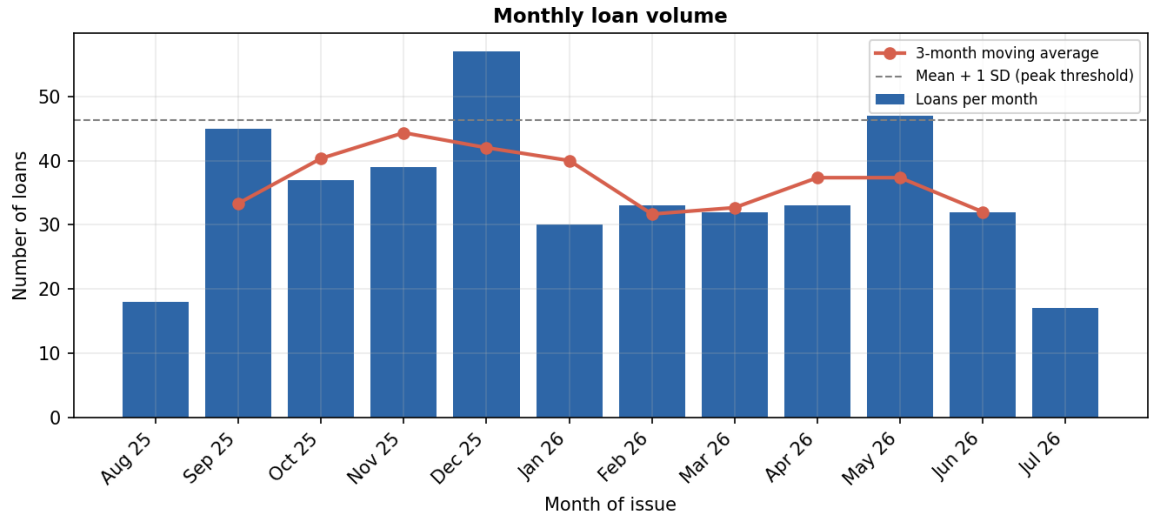


Figure 1. Loans issued per month with a 3-month moving average and the peak threshold.

6.3 Demand by category

Category	Loans	Share	Borrowers	Mean days	Overdue	Titles	Loans/title
Computer Science	102	24.3%	55	11.5	24.5%	60	1.70
Electronics	101	24.0%	53	11.0	27.7%	50	2.02
Mathematics	84	20.0%	42	11.0	21.4%	40	2.10
Literature	50	11.9%	33	10.9	20.0%	40	1.25
Physics	47	11.2%	33	10.9	23.4%	30	1.57
Management	36	8.6%	26	11.2	25.0%	20	1.80

Computer Science (102 loans) and Electronics (101) are practically tied at the top; a difference of one loan is not meaningful. Mathematics follows with 84. Literature, Physics and Management each account for between 8.6% and 11.9% of loans.

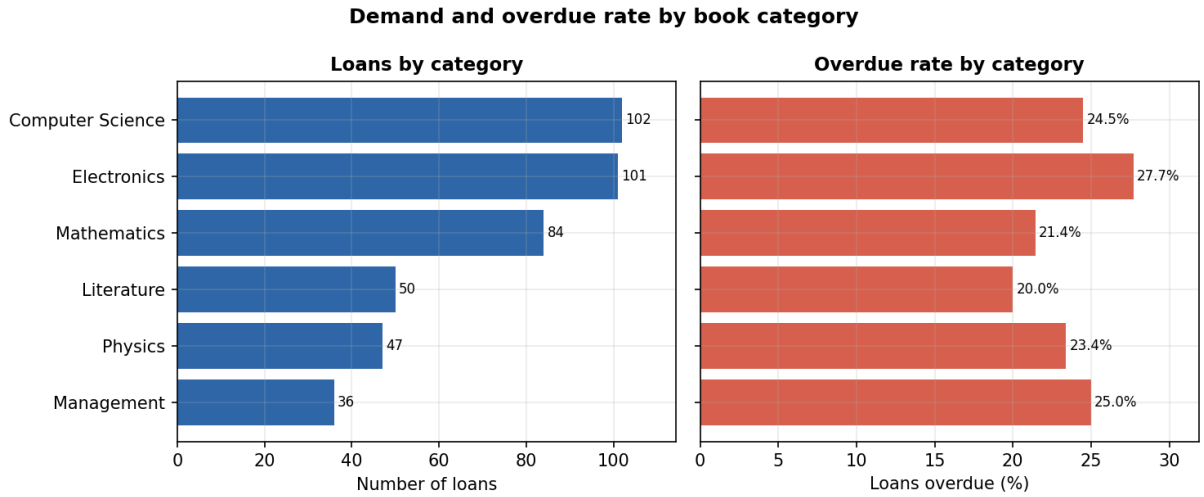


Figure 2. Loans (left) and overdue rate (right) by book category.

6.4 Loan lengths

For the 417 returned loans the mean length is 11.1 days and the median 11 days; 90% of loans are no longer than 17 days and the longest is 25 days. The due date is 14 days after issue, so the distribution straddles the due date and a substantial minority of loans is overdue.

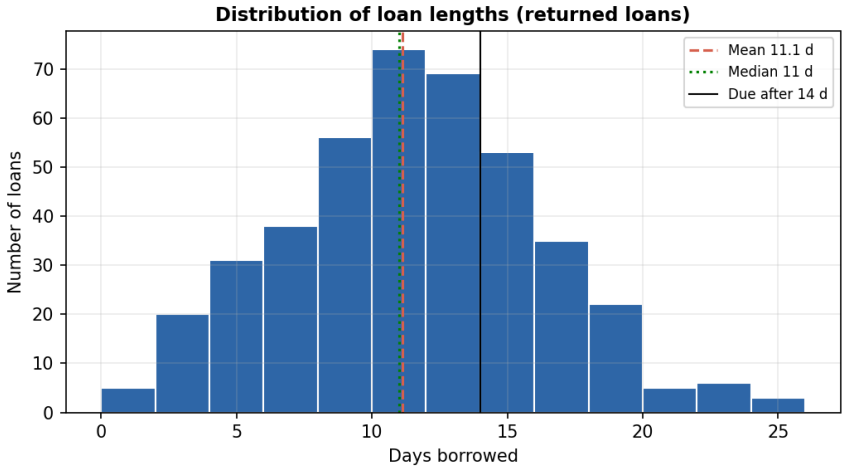


Figure 3. Distribution of loan lengths for returned loans.

6.5 Evaluation of alternatives

(A) Raw versus cleaned data. Counting every raw row inflates the counts of the categories that contain duplicated or incomplete rows, by up to 6.0%. The ranking of the six categories does not change in this dataset, but a close contest (such as the top two) could be decided by faulty rows, so cleaning is needed before any ranking is reported.

(B) Count versus loans per title. By raw count the order is Computer Science, Electronics, Mathematics ...; per catalogue title the order is Mathematics, Electronics, Management ... Computer Science falls from first to rank 4 because it has the largest collection (60 titles). Neither measure is wrong: count shows total workload, while loans per title shows how intensively each title is used. A purchasing decision should consider both. Because the catalogue file lists titles, not physical copies, loans per title is only an approximation of use per copy.

(C) Overdue rule. With the stored 14-day rule 24.0% of loans are overdue, but with a 7-day grace period only 2.1% are. The overdue rate is therefore very sensitive to the rule chosen and should always be reported together with the rule.

Category	Raw rows	Cleaned	Inflation	Rank by count	Rank per title	Overdue 0-day	Overdue 7-day
Computer Science	106	102	3.9%	1	4	24.5%	3.9%
Electronics	104	101	3.0%	2	2	27.7%	1.0%
Mathematics	88	84	4.8%	3	1	21.4%	0.0%
Literature	53	50	6.0%	4	6	20.0%	4.0%
Physics	47	47	0.0%	5	5	23.4%	2.1%
Management	37	36	2.8%	6	3	25.0%	2.8%

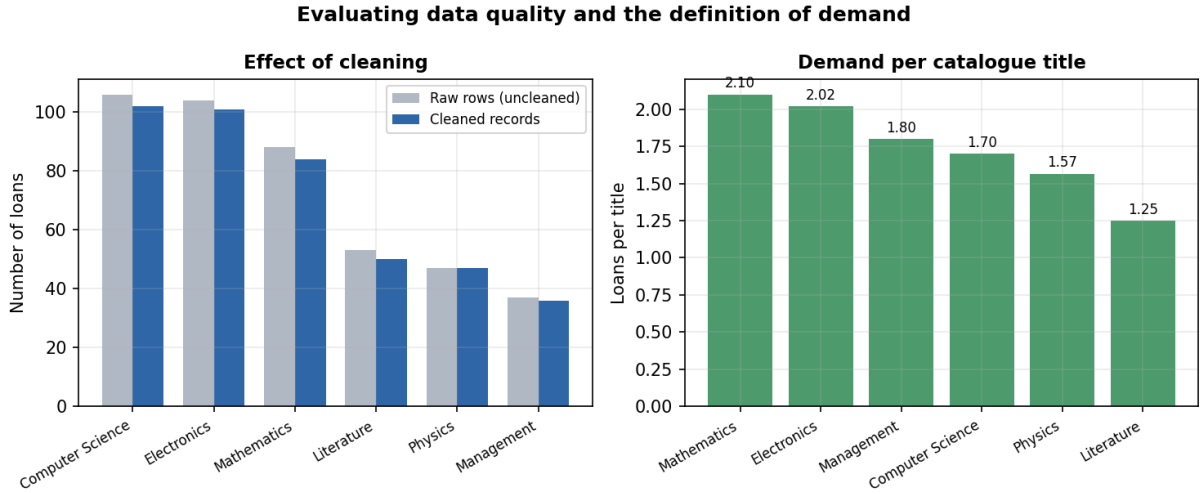


Figure 4. Effect of cleaning (left) and demand per catalogue title (right).

6.6 Interpretation

- The data cleaning step removed 17 of 437 rows (3.9%), and every injected fault was found.
- High demand has two meanings here. Computer Science and Electronics have the largest total demand; Mathematics has the largest demand per catalogue title.
- Peak months should be checked against the academic calendar before being treated as real demand.
- The overdue percentage depends strongly on the rule and grace period used.
- The dataset is simulated and the demand weights, exam months and fault rows were chosen by the authors; the results show that the dashboard works, not what real students borrow.

7. Testing and Validation

The file *tests.py* contains 23 automated unit tests (Python *unittest*); all pass. The table summarises what is checked.

Test case	Expected behaviour	Observed
Loan returned after due date	Marked overdue	Pass
Loan returned exactly on due date	Not overdue	Pass
Loan still out, due date passed	Overdue at the reference date	Pass
7-day grace period	Loan 4 days late is not overdue	Pass
Impossible date (31/02/2026)	Rejected as invalid date format	Rejected

Test case	Expected behaviour	Observed
Blank mandatory field (each of six)	Rejected as missing field	Rejected
Return or due date before issue date	Rejected	Rejected
Blank return date	Accepted as still on loan	Accepted
Duplicate loan_id	Second occurrence rejected; line number correct	Rejected
Unknown category	Rejected	Rejected
CSV with a missing column	Error raised with a clear message	Pass
Special methods __eq__, __hash__, __lt__, __repr__	Behave as documented	Pass
LoanCollection len, indexing, slicing, iteration	Correct values; slice returns a collection	Pass
NumPy durations and moving average	Correct array values and length	Pass
Full run on the sample data	Rows add up; category and monthly totals equal valid records; all output files written	Pass
Grace-period comparison	Overdue rate with 7-day grace is lower than without	Pass

Result of *python tests.py*: **Ran 23 tests - OK**. The program's console output and the test log are saved in the *outputs* folder of the repository. Plot appearance was checked by eye; the tests check that the figure files are created.

8. Limitations and Future Scope

Limitations: (i) the data are simulated and cover one library for one academic year; (ii) demand is measured by loans, which does not capture unmet demand such as waiting lists or reading inside the library; (iii) loans per title uses titles, not physical copies; (iv) the peak rule (mean + 1 SD) and the 14-day loan period are fixed assumptions; (v) duplicates are detected by *loan_id* only, so the same loan entered under two different IDs would not be caught; (vi) the program is non-interactive and has no user accounts or access control.

Future scope: a graphical or web interface; real (consented, anonymised) library data; copies per title; detection of near-duplicate loans; forecasting of monthly demand; handling of renewals and fines; and a comparison of several anomaly rules. Any real borrowing data should be anonymised and handled with privacy safeguards.

9. Conclusion

The project delivers a complete, reproducible workflow for analysing library circulation: file handling, validation, classes with special methods, NumPy statistics, Pandas summaries and JSON export, and Matplotlib figures with a written interpretation. On the simulated data the pipeline detected all 17 faulty rows, identified Computer Science and Electronics as the categories with the most loans, Mathematics as the category with the highest demand per title, and December and May as peak months. The central analytical conclusion is cautious: what counts as "high demand" and what counts as "overdue" depend on the chosen definitions and on the quality of the records, so these choices must be stated with every result.

10. References

Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/

Python Software Foundation. *csv, json, datetime and unittest modules*. https://docs.python.org/3/library/

NumPy Developers. *NumPy Documentation*. https://numpy.org/doc/

Pandas Development Team. *pandas Documentation*. https://pandas.pydata.org/docs/

Matplotlib Development Team. *Matplotlib Documentation*. https://matplotlib.org/stable/

11. Appendix: CSV Format, Execution and Contributions

11.1 CSV format

Required headers of the loan file (exact spelling): *loan_id, student_id, book_id, category, issue_date, due_date, return_date*. Dates use YYYY-MM-DD; a blank *return_date* means the book is still on loan.

loan_id	student_id	book_id	category	issue_date	due_date	return_date
L0001	M006	B158	Physics	2026-03-01	2026-03-15	2026-03-11
L0002	M001	B113	Mathematics	2026-06-03	2026-06-17	2026-06-07

11.2 Running the program

1. Install the libraries: `pip install numpy pandas matplotlib`
2. (Optional) regenerate the simulated data: `python data_generator.py`
3. Run the analysis: `python library_dashboard.py` (add `--show` to open the plots)
4. Run the tests: `python tests.py`

The complete source code, data, outputs and README are in the project's GitHub repository (*Library-Circulation-Demand-Dashboard*).

11.3 Individual contributions

Name	Roll no.	Contribution
Kushal Chakraborty	58	Individual project: design, coding, data generation, testing, analysis and report